The Complete Wayland Handbook

A Comprehensive Deep Dive into Compositors, Architecture, and Customization

---

I. Understanding the Paradigm Shift

To understand Wayland, one must first appreciate the history of Linux graphics. For over 30 years, the **X Window System (X11)** reigned supreme. It was built on a "Network Transparent" model where a central server (Xorg) acted as a massive middleman between your apps and your hardware.

The Problem with Xorg

Xorg suffers from what developers call "The Telephone Game." When you click a mouse button:

1. The Kernel catches the click and sends it to the X Server.
2. The X Server asks the Window Manager (like i3 or Openbox) where the window is.
3. The X Server tells the app (like Firefox) that it was clicked.
4. The app draws a new frame and sends it back to the X Server.
5. The X Server sends it to a *Compositor* (like Picom) to add shadows/transparency.
6. The Compositor sends it back to the X Server to finally show you.

This complexity leads to **Input Lag** and **Screen Tearing**, as various parts of the screen update at different times.

The Wayland Solution

Wayland removes the middleman. In Wayland, the **Compositor IS the Server.** It is the single source of truth for everything on your screen. This results in a "Every Frame is Perfect" design philosophy.

II. Under the Hood: Technical Architecture

Wayland is not a program; it is a **protocol**. It defines how an application and a compositor talk to each other. Here are the core technical components that make it work:

1. DMA-BUF (Direct Memory Access Buffers)

In Xorg, image data was often copied from the app to the server—a slow process. Wayland uses **Shared Memory**. An app draws into a buffer in the GPU's memory and simply hands a "file descriptor" (a digital claim ticket) to the compositor. The compositor looks at the memory directly. No copying, no delay.

2. KMS (Kernel Mode Setting)

The compositor talks directly to the Linux Kernel via KMS. This is the low-level system that tells the monitor what resolution to use and what pixels to display. By removing the X Server layer, the path from the app to the monitor is significantly shortened.

3. libinput & evdev

Wayland compositors use `libinput` to handle mouse and keyboard events directly from the kernel. This allows for modern features like smooth touchpad gestures (pinch-to-zoom, multi-finger swipes) that were notoriously difficult to implement in X11.

III. The "Lego" Desktop Philosophy

When you use a Desktop Environment like GNOME or KDE, everything is built for you. However, the true power of Wayland lies in **Standalone Compositors**. This is often called "Ricing." You choose each individual "brick" to build your own environment.

The Core Bricks

Component	Job Description	Popular Wayland Choice
Compositor	Manages windows, input, and rendering.	Hyprland, Sway, River
Status Bar	Displays clock, battery, and workspaces.	Waybar
App Launcher	The "Search" bar to find and open apps.	Wofi, Rofi-wayland
Notification Daemon	Displays popups from apps.	Mako, Swaync
Wallpaper Utility	Puts an image (or GIF) on the background.	swww, Hyprpaper

IV. Desktop Portals: The Secure Bridge

Wayland is secure by design. Apps are **sandboxed**, meaning Firefox cannot "see" your terminal, and OBS cannot "see" your screen without permission. **Desktop Portals** are the standard way apps ask for these permissions.

The Specialist vs. The Assistant

In a custom setup, you typically need at least two portals working together:

- **The Specialist (e.g., XDPH):** `xdg-desktop-portal-hyprland` is built to handle high-performance tasks like screen sharing and screenshots for the Hyprland engine.
- **The Assistant (GTK):** `xdg-desktop-portal-gtk` provides the actual "File Open" windows and system settings that the compositor itself doesn't have.

Configuring Portals

You must tell the system which portal to use for which task in your `portals.conf` file:

```
# Location: ~/.config/xdg-desktop-portal/portals.conf
[preferred]
# Use Hyprland for screencasting (performance)
org.freedesktop.impl.portal.ScreenCast = hyprland
org.freedesktop.impl.portal.Screenshot = hyprland

# Use GTK for file picking and settings (UI)
org.freedesktop.impl.portal.FileChooser = gtk
org.freedesktop.impl.portal.Settings = gtk

# Default fallback
default = hyprland;gtk
```

> **Troubleshooting:** If an app freezes for 30 seconds when opening a file, it means it is waiting for a portal that isn't running or isn't configured in this file.

V. GSettings: The Global Style Sheet

GSettings is the central database where your preferences (fonts, themes, cursors) live. Without setting these, your apps will look inconsistent—some might be light, some dark, and your cursor might change size randomly.

Essential GSettings Keys

- `gtk-theme`: Sets the overall look of windows.
- `cursor-theme`: Essential to prevent the cursor from changing shape between windows.
- `cursor-size`: Should be set to a fixed value (e.g., 24).
- `color-scheme`: Set to `'prefer-dark'` to trigger Dark Mode in modern apps.

The Detective Method: How to Find Keys

If you aren't sure which setting controls a specific feature, use the **Spy Method**:

1. Open a terminal and type: `gsettings monitor`
2. Open an app's preferences and change a setting.
3. The terminal will immediately print the **Schema** and **Key** that changed.

Implementation in Config

Since standalone compositors don't have a settings manager, you should "force" these settings in your startup config:

```
# In your hyprland.conf or sway config
exec-once = gsettings set org.gnome.desktop.interface gtk-theme 'Nordic'
exec-once = gsettings set org.gnome.desktop.interface cursor-size 24
exec-once = gsettings set org.gnome.desktop.interface color-scheme 'prefe-dark'
```

VI. Triggering Portals from Inside Apps

Apps don't "know" about portals; they use libraries like GTK or Qt. You can force these libraries to use Wayland protocols via environment variables:

```
# Force GTK apps to use the portal for file picking
env = GTK_USE_PORTAL,1

# Force Qt apps to use Wayland instead of X11
env = QT_QPA_PLATFORM,wayland
```

When you click "Upload" in a browser, it sends a **D-Bus** message. The `xdg-desktop-portal` daemon hears this, checks your `portals.conf`, and launches the appropriate window (like the GTK file picker) to fulfill the request.